

Characterizing Dataflow for I/O-Aware Scheduling in HPC Workflows

Meng Tang¹, Luanzheng Guo², Anthony Kougkas¹, Xian-He Sun¹, Nathan R. Tallent²

¹Illinois Institute of Technology, ²Pacific Northwest National Laboratory

Emails: mtang11@hawk.illinoistech.edu; lenny.guo@pnnl.gov; akougkas, sun@illinoistech.edu; tallent@pnnl.gov;

Abstract—Scientific discoveries are increasingly facilitated by data-driven workflows executed in HPC environments, where complex task-to-data exchange patterns cause bottlenecks in shared storage. Existing I/O-aware scheduling techniques use application-oriented metrics that primarily focus on individual task characteristics, leaving workflow-level dataflow behaviors, such as file reuse and producer-consumer I/O patterns, underexplored despite their importance for effective orchestration. This paper presents a workflow-centric methodology to characterize and analyze I/O using workflow-specific patterns and metrics. By correlating workflow structures (e.g., task composition, execution order, and producer-consumer relationships) with I/O metrics (e.g., access type, operation count, and data size), we gain deeper insight into workflow I/O dynamics. We evaluate our approach on six diverse HPC workflows, covering 3,500 task instances and 1,387 unique files, with a total I/O volume of 1.5 TB. This analysis uncovers a spectrum of behaviors related to file reuse, data exchange, and task-level I/O heterogeneity. These insights reveal new opportunities and research directions for I/O-aware scheduling, supporting smarter task placement, data movement, and storage selection in HPC.

I. INTRODUCTION

Scientific discovery has increasingly become data-centric, driven by advancements in high-throughput instruments, large-scale simulations, and AI-assisted workflows. As artificial intelligence begins to play a central role in scientific processes—ranging from instrument control and workflow orchestration to data analysis and hypothesis generation—the volume and complexity of data movement within scientific computing have grown drastically [1], [2], [3]. This trend is now common across a wide spectrum of scientific disciplines [4], [5], [6]. Consequently, the computing paradigm in HPC has shifted from compute-centric to data-intensive, where memory and I/O performance are increasingly becoming limiting factors. These workflows today comprise multiple interconnected applications—including simulation, analysis, and AI—exchanging data in producer-consumer patterns that introduce complex inter-task data dependencies [7], [8], [9]. This data is typically transferred through shared parallel file systems (PFS), which often suffer from slow access and contention [10], leading to serious I/O bottlenecks [11], [12].

To address these challenges, recent research has proposed a range of I/O-aware scheduling approaches, including dynamic coordination of I/O and compute [13], [14], graph-based task placement [15], [16], and AI-assisted scheduling decisions [17]. Despite these advances, a fundamental gap remains: current techniques often lack a structural under-

standing of I/O behavior within workflows. Existing profiling tools typically focus on system-level metrics or isolated tasks, missing broader workflow-level patterns—such as file reuse, stage ordering, and producer-consumer dataflows—that are essential for I/O-aware scheduling.

To bridge this gap, this paper introduces a comprehensive methodology for capturing, characterizing, and visualizing task-file I/O relationships within complex HPC workflows. By linking high-level workflow structures (such as tasks, stages, files, and producer-consumer groups) with low-level I/O metrics including bandwidth, operation count, dataflow size, and access type, we reveal how data flows between tasks in relation to I/O dynamics. Grouping I/O behavior by task groups and file types exposes patterns of file reuse, access asymmetry, and performance bottlenecks that are often hidden in traditional system-level I/O analyses. This workflow-centric perspective enables us to both validate known I/O challenges and uncover previously unreported patterns, offering new insights for intelligent, workflow-aware I/O scheduling and performance optimization.

Our contributions are as follows:

- We present a **workflow-centric methodology** to characterize, visualize, and analyze I/O behavior in HPC environments. Our approach correlates workflow context—such as task composition, execution stages, and producer-consumer relationships—with low-level I/O metrics including dataflow size, access type, and operation count.
- We conduct a holistic analysis of six HPC workflows, applying our methodology to identify and classify common **producer-consumer dataflow patterns**. We characterize these patterns using observed bandwidth, I/O characteristics, and access types, revealing performance bottlenecks across various workflow stages and data exchange patterns.
- We uncover 16 workflow I/O-specific findings and identify opportunities for optimization in areas such as file reuse and producer-consumer communication. These insights expose limitations in current workflow execution models and provide guidance for improving I/O-aware scheduling, task placement, and storage selection in HPC systems.

II. METHODOLOGY

A. HPC Workflows

HPC workflows are structured as tasks organized into stages, with data dependencies represented as a *Directed Acyclic Graph (DAG)*. This structure is key to analyzing inter-task dependencies and scheduling requirements. Our study includes six representative workflows from genomics, climate science, astronomy, seismology, molecular dynamics, and machine learning. They were selected for their diverse scale, structure, and I/O complexity, as summarized in Table III.

B. Dataflow Pattern Characterization

Characterizing producer–consumer (P–C) I/O patterns is essential for understanding data movement at a workflow level. Our methodology is built upon two key conceptual units: **Task Groups** and **Producer-Consumer (P–C) relationships**. We define a Task Group as a set of tasks that share the same executable and file access patterns. P–C relationships are then established by matching the output files from a producer task group with the input files of a consumer task group. By identifying these units from I/O trace data, we can classify the resulting dataflows using our taxonomy and analyze behavior at a higher level of abstraction than individual file operations. Our P–C pattern taxonomy (Table I) systematizes concepts from established workflow and parallel computing literature—such as ‘scatter-gather’ from Pegasus [18] and foundational parallel design patterns [19]—into a formal, I/O-centric classification. Our novelty lies in abstracting these into a concise set (1-1, 1-n, n-1, n-n) that can be used to characterize I/O dataflows uniformly. These core patterns can combine to form broader workflow structures (Table II), and are critical for identifying bottlenecks and informing I/O-aware scheduling.

C. Data Collection and Profiling

The analysis in this paper relies on detailed I/O traces. To capture them, we use the Datalife [20] tool for runtime monitoring. This non-intrusive approach uses *LD_PRELOAD* to intercept POSIX-level I/O calls for each workflow task without modifying source code. This passive instrumentation generates per-process trace files containing detailed metrics such as I/O operation type, data size, and operation count, with a measured overhead of less than 0.5% in execution time [20].

D. Characterization and Visualization

Characterization: Understanding dataflow for workflow scheduling requires analyzing I/O patterns in the context of the workflow—specifically, how data moves between tasks and how I/O behavior varies across producer–consumer (P–C) groups. These connections reveal key demands and bottlenecks that are crucial for effective I/O-aware scheduling in data-intensive workflows.

To quantify the behavior of the P–C patterns and their components, we measure the following metrics:

- 1) **Dataflow Producer–Consumer Group:** A foundational metric that captures the data dependency between two

groups of tasks: the producer group (writing) and the consumer group (reading). Explicitly modeling the producer–consumer (P–C) relationship provides essential workflow-level context for uncovering I/O behaviors during data exchange and offers insights into data placement and task management.

- 2) **Functional Workflow Stage:** Workflows typically execute in stages, each representing a logical unit of work in the pipeline (e.g., preprocessing, training). Stage order captures the temporal sequence of tasks and identifies potential I/O contention along the dataflow lifecycle.
- 3) **Transferred Dataflow:** The total amount of data transferred between a task and a file. This value reflects the actual I/O volume, which may differ from the file size.

In addition to these structural metrics, we incorporate several standard I/O characterization metrics into our analysis. These include grouping files by type (**File Group**), grouping parallel tasks from the same application by type (**Task Group**), and measuring performance indicators such as effective **I/O Bandwidth**, total **Operation Count**, and the access **I/O Type** (e.g., sequential or random reads and writes).

Multidimensional I/O Visualization. To provide a structural view of complex I/O statistics in the dataflow lifecycle, we propose multidimensional dataflow visualization tools based on three core components:

1. Workflow Context: This is the foundational principle for examining I/O behavior from different analytical perspectives. We apply groupings by *Producer–Consumer Groups*, *Task Groups*, *File Groups*, and *I/O Operation Types*.

Given a context, there are two basic views:

2. Stage-centric View: This plot view captures temporal I/O hotspots by visualizing Workflow Stage $\times$ Bandwidth $\times$ Dataflow Size.

3. Dataflow-centric View: This plot view captures dataflow-level I/O intensity by visualizing Operation Count $\times$ I/O Bandwidth $\times$ Dataflow Size.

Our typical analysis workflow follows a systematic order: we first examine producer–consumer plots to identify bottleneck areas in data exchange. We then use workflow context to investigate these bottlenecks, starting with task groups to identify the involved tasks, followed by file groups to identify the involved files. The stage-centric view allows us to observe temporal information and I/O hotspots across workflow stages, while the dataflow-centric view provides pattern-wise insights into I/O characteristics such as operation intensity and bandwidth distribution.

By employing these components, we gain critical insights into dataflow dependencies. For instance, Figure 3b shows an *operation-centric 3D plot grouped by producer–consumer (P–C) groups*. In this view, operation count and dataflow are summed across all tasks within each P–C group, while bandwidth is aggregated separately for the producer and consumer tasks before being averaged. The legend indicates the associated P–C tasks and files used for data transfer. For clarity, we prepend *initial* to files read without a preceding task, and append *final* to files last used in the workflow. In

Pattern	Description	Example Tasks
1-1	One producer writes data read by one consumer.	A simulation outputs results used by a single postprocessing step.
1-n	One producer’s data is shared across multiple consumers.	Static input or checkpoint file used by many analysis tasks.
n-1	Several producers write parts consumed by one task.	Aggregation or merging of outputs from parallel tasks.
n-n	Multiple producers and consumers with shared I/O.	Parallel ML training or ensemble workflows.

TABLE I: Core producer–consumer I/O patterns in workflow interactions.

Workflow Pattern	Description	Producer–Consumer Structures
Cascading	Tasks pass outputs to the next stage in a linear chain.	Can include all patterns in Table I.
Iterative	Repeats computation with prior output as input.	Often [1-1] ; may include [1-n] , [n-1] , or [n-n] .
Conditional	Tasks access data only if certain conditions are met.	Usually [1-1] or [1-n] with conditional logic.
Checkpointing	Saves task state for restart or recovery.	Usually [1-1] ; may be [n-1] if consolidated.
Broadcast	One task provides shared input to many others.	Typical [1-n] pattern.
Scatter-Gather	Data is distributed to parallel tasks, then merged.	[1-n] for scatter; [n-1] or [n-n] for gather.

TABLE II: Workflow-level I/O patterns and corresponding producer–consumer structures.

Workflow	# Stages	# Task Instances	Total I/O Size (GB)	# Unique Files	Avg. File Reuse	P-C Patterns	Workflow-level I/O Patterns
PyflexTRKR	8	774	11.54	197	4.68	n-n, 1-n, 1-1	Cascading, Scatter-Gather
1000 Genomes	3	1340	1445.74	481	3.74	n-n, 1-n, n-1, 1-1	Cascading, Scatter-Gather
Seismology	2	478	0.51	480	1.17	n-1, 1-1	Gather
Montage	8	808	17.15	156	6.05	n-n, n-1, 1-1	Cascading, Scatter-Gather
DeepDriveMD	2	74	5.48	50	1.78	n-1, 1-1	Checkpointing, Iterative, Conditional
PtychoNN	3	25	2.60	18	1.72	1-1, n-1	Checkpointing, Iterative, Conditional

TABLE III: Comparison of I/O and structural characteristics across diverse HPC workflows.

contrast, Figure 1e presents a *stage-centric 2D plot grouped by file groups*, where each data point’s area reflects cumulative task-to-file dataflow, and color depth indicates overlapping density.

E. Experimental Setup

We evaluated a diverse set of six HPC workflows, comprising 3,500 tasks and capturing a total of 1.5 TB of exchanged I/O data. Our analysis covers 1,387 unique files spanning 17 widely-used file formats in HPC and data-intensive domains (Table IV). This coverage reflects a wide range of real-world I/O access patterns and storage demands across multiple scientific domains.

The evaluation was conducted on the machines listed in Table V. While our experiments are conducted at small-to-medium scales, the identified structural I/O patterns are inherent to the workflow’s design and thus largely scale-independent, as demonstrated in prior work [21]; while quantitative metrics like absolute bandwidth vary with system size, the qualitative patterns remain consistent across scales. The focus of this work is this foundational characterization, which is complementary to performance modeling that predicts how metrics change with scale. Smaller deployments, where many control loops in scientific workflows naturally operate, often highlight optimization opportunities by reducing data movement and resource contention [22].

Specifically, PyFlexTRKR was executed over 12 nodes, 1000 Genomes over 10 nodes, DeepDriveMD over 4 nodes, and the remaining workflows ran on a single node. An additional large language model (LLM) workflow was evaluated

but not presented, as its findings were similar to those of the other two machine learning workflows.

Each of the six workflows was executed 3 times to ensure reproducibility. The observed structural I/O patterns and performance trends were consistent across all runs, and the standard deviation in key performance metrics was recorded. For clarity, results presented in the following sections are from a single representative run of each workflow. The I/O traces and analysis scripts used to generate the figures in this paper, along with additional 2D and 3D visualization plots, are available at <https://github.com/candiceT233/dataflow-for-io-aware>.

Category	File Formats and Domains
Scientific / Domain-Specific	HDF5, netCDF4, FITS, DCD, TBL, LHT, STF, VCF — used across earth, space, and life sciences
ML / Array	NPY, NPZ, PT, safetensors — Python-based deep learning workflows (e.g., PyTorch, TensorFlow)
Compressed / Standard	GZ, JSON, TXT, PNG, PDF — general data, meta-data, images and text

TABLE IV: File format categories and usage domains.

III. HPC WORKFLOW I/O CHARACTERISTICS

We distinguish between structural workflow properties (inherent to design: P-C patterns, file reuse, stage organization) and execution parameters (deployment-specific: parallelism, node count, storage configuration) to assess whether bottlenecks require workflow redesign or deployment tuning.

A. Seismology

The Seismic Cross-Correlation workflow is a data-intensive seismology application used to analyze ground motion data

Component	Configuration / Options
CPU Cluster	2x Intel Xeon Silver 4114 (20 cores), 48 GB RAM
GPU Cluster	2x AMD EPYC 7502 (64 cores), 944 GB RAM
Storage	NFS (global default); Node-Local: NVMe SSD, SATA SSD, HDD (CPU); 224 GB SATA SSD (GPU)

TABLE V: Machine configurations for experiments.

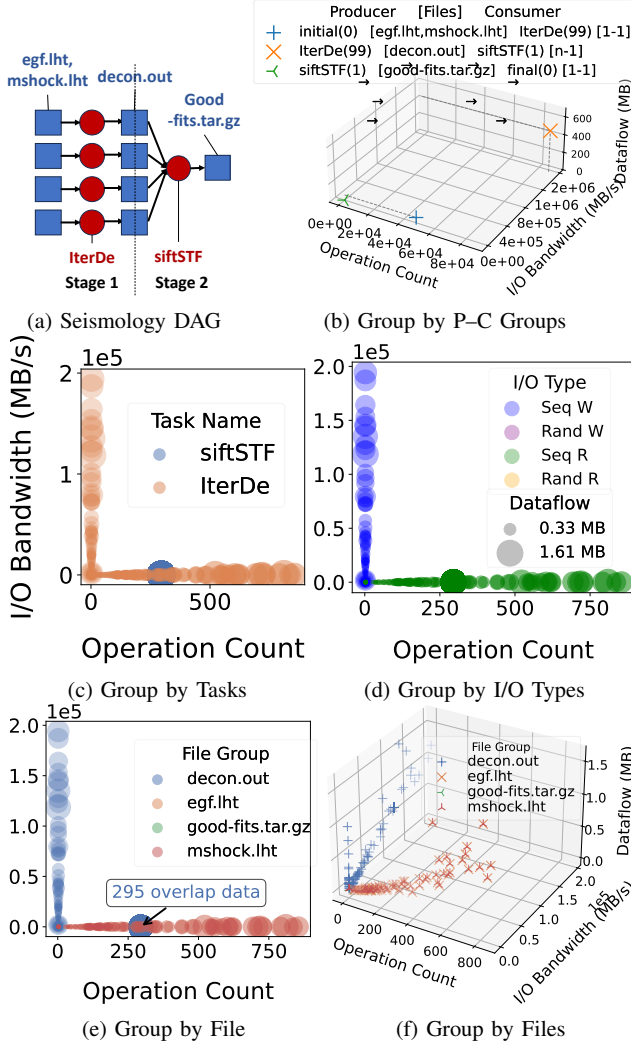


Fig. 1: Seismology Workflow: structural characterization and visualization of I/O profiles.

from multiple seismic stations [23]. It consists of two main tasks: *sG1IterDecon*, which takes pairs of seismic signals and performs deconvolution to estimate source time functions (STFs); and *siftSTFByMisfit*, which filters and compresses the resulting STFs based on quality. The workflow is representative of result aggregation from many parallel tasks.

1) Observation:

Figure 1 presents the characterization of I/O activity in the Seismology workflow across various grouping dimensions. To provide context on task structure, the workflow DAG is shown in Figure 1a. Seismology consists of two main task groups, with *IterDe* as the producer group and *siftSTF* as the consumer

group. In this workflow, we see *Gather* I/O patterns.

Finding 1: Write performance variation under high parallelism. Combining the plots (Figures 1c, 1e, and 1d), we observe a significant variation in I/O performance for write operations, even though the overall trend lines for a specific file type remain consistent. We first examine the tasks using Figure 1c. The data points for the *IterDe* task (orange) exhibit two distinct behaviors: (1) a similar operation count but a wide range of bandwidths and (2) a wide range of operation counts but a similar, low bandwidth. By mapping these data points onto the corresponding locations in Figure 1d, we identify that the similar operation count with wide bandwidth variation corresponds to sequential write operations (blue), while the wide operation count with similar bandwidth corresponds to sequential read operations (green).

Next, Figure 1e reveals that all these sequential write operations are linked to the single file *decon.out* (also shown in blue). Figure 1b provides the context: this workflow involves *IterDe* as the main producer and *siftSTF* (highlighted in orange) as the main consumer, with *IterDe* running under high parallelism (99 concurrent tasks on a single node). This variation in write performance, despite targeting the same file, indicates that the 99 parallel *IterDe* tasks running concurrently are causing resource contention. This deployment configuration exceeds available CPU cores, creating resource contention particularly problematic for write operations. This observation demonstrates that write operations are sensitive to resource contention under excessive parallelism, and that adjusting the deployment configuration (e.g., distributing tasks across more nodes or reducing concurrent tasks per node) could mitigate this bottleneck.

Finding 2: Intra-task variation in I/O patterns and performance. I/O behavior can vary significantly within a single task. In contrast to the variable write performance observed in task *IterDe* (Finding 1), Figure 1e shows that all its read operations on *egf.lht* and *mshock.lht* form compact clusters, maintaining consistently low performance. Furthermore, Figure 1f provides a 3D view of this data, illustrating the linear relationship between this low performance and the data flow. This observation suggests a bottleneck distinct from the contention seen in writes and implies that improving read access in these cases could help enhance overall I/O efficiency.

B. 1000 Genomes

The 1000 Genomes project [24] is a data-intensive bioinformatics workflow. It processes multiple chromosomes concurrently using five distinct tasks, *indiv*, *merge*, *sift*, *mutat*, and *freq*, each exhibiting different access patterns of data splitting and aggregation. Each chromosome’s biological signature is stored in a separate file, and multiple *indiv* tasks process these files concurrently, each handling a unique segment of the data. The *merge* task then aggregates all input files into a single compressed file for each chromosome for later processing.

1) Observation:

Figure 2 presents the characterization of I/O activity in the 1000 Genomes workflow across various grouping dimensions.

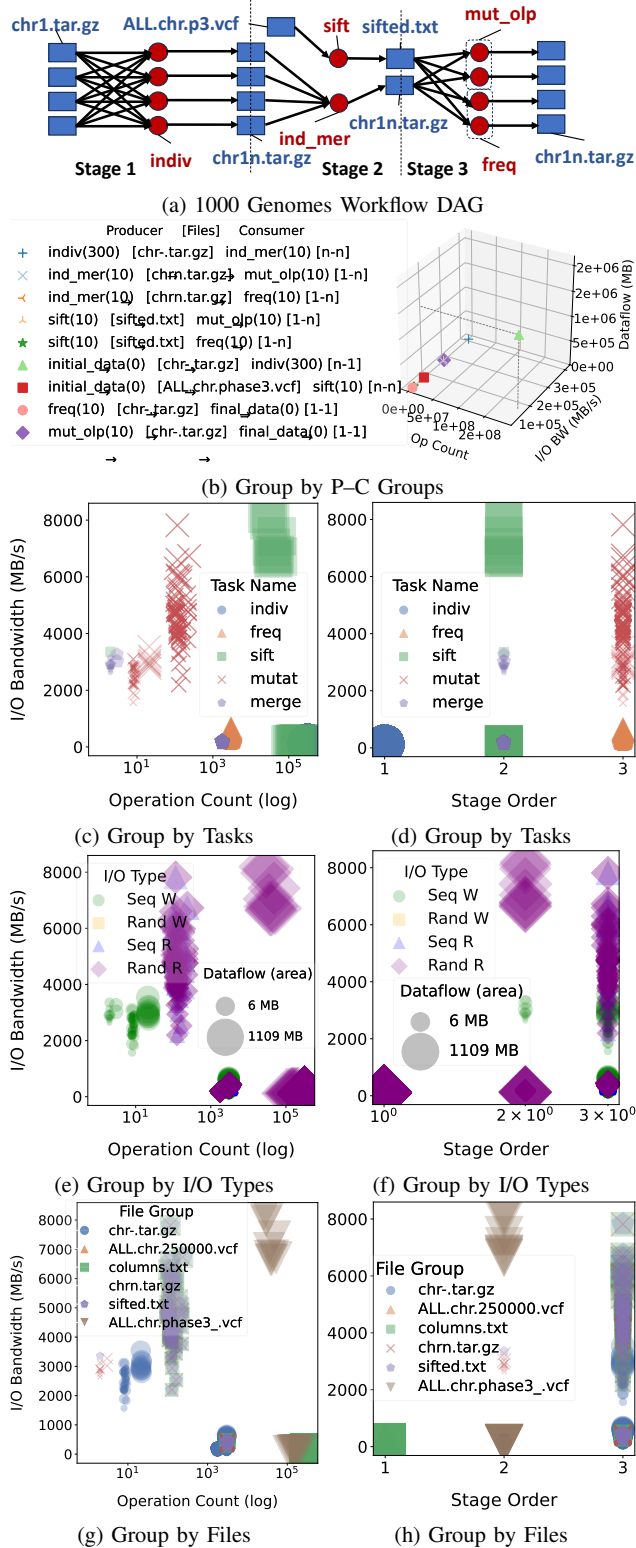


Fig. 2: Structural characterization and visualization of I/O profiles for the 1000 Genomes workflow.

This workflow achieves an average bandwidth of up to 1 GB/s. Figure 2a illustrates the DAG for Chromosome 1 (*chr1*), as the full DAG across all 10 chromosomes would be too large to display. The diagram highlights key dependencies (such as those between the *indiv* and *merge* tasks) while showing that *sift* operates independently, and that both *freq* and *mutat* depend on outputs from *merge* and *sift*. This workflow exhibits Broadcast and Scatter-Gather I/O patterns.

Finding 3: Cached Read Performance Improvement. Reading a cached file from memory provides significantly better performance than reading from shared storage. We first use Figure 2d to identify the *sift* task (red triangle) in Stage 2 as exhibiting two distinct performance clusters. Mapping these points to Figure 2f confirms they are random read operations (yellow). Finally, Figure 2h shows both clusters link to the same file *ALL.chr.phase3.vcf* (brown cross) despite similar dataflow volume. This variation arises because the task performs multiple reads: the initial access achieves only 126 MB/s, while subsequent reads, benefiting from caching, reach up to 7.4 GB/s. A similar performance improvement pattern is observed for task *mutat* (green spade) when reading files *sifted.txt*, *columns.txt*, and *chrn.tar.gz*.

Finding 4: Inter-stage variation in I/O patterns and performance. I/O behavior varies significantly across workflow stages, emphasizing the dynamic nature of I/O demands over time. We first use Figure 2f to observe that Stage 3 generates substantially more I/O activity across a wide range of throughput, while Stage 1 exhibits the lowest throughput. The operation types responsible for this Stage 3 activity are primarily sequential writes and random reads. Next, Figure 2h confirms that both the number of files accessed and the total volume of data transferred also vary significantly between stages. Understanding this temporal sequencing of stages enables better forecasting of I/O demands—including access types, data volume, and concurrency—which is critical for workload planning and storage optimization.

Finding 5: Performance difference due to distinct I/O patterns on shared files. When different tasks interact with the same files using distinct I/O patterns, significant performance differences emerge. We first use Figure 2c to examine tasks *freq* (orange square) and *mutat* (green spade). Referencing Figure 2e, we confirm both tasks involve read and write operations, but with contrasting behavior: *freq* performs many small operations (ave. <1 KB), yielding lower throughput, while *mutat* shows lower operation counts but broader throughput variability (suggesting larger or more sequential operations). Figure 2g confirms both tasks access the same files: *sifted.txt* and the grouped *chr.tar.gz*. This difference in I/O pattern is the primary driver of performance variation. Tasks like *freq* that perform many small, fragmented operations experience lower bandwidth due to overhead and access inefficiency. Understanding this functional divergence over shared data is essential for designing access-aware storage strategies. Possible optimizations include using storage systems with efficient support for small random I/O, such as SSD-based tiers or memory-backed file systems, to mitigate

this performance degradation.

C. PyFlexTRKR

We study PyFlexTRKR [25], an analysis-phase framework for storm tracking in atmospheric science. PyFlexTRKR employs feature tracking algorithms for atmospheric data and is organized into nine sequential stages, with parallel task execution managed by a central workflow orchestrator script. The early stages perform feature identification and mapping, while the later stages focus on data-intensive feature analysis. Its complex dependencies and irregular I/O patterns, which are typical of feature-based image analysis workflows, make it a representative case for studying I/O behaviors in HPC workflows.

1) Observation:

Figure 3 presents the structural I/O behavior of the PyFlexTRKR workflow across various grouping dimensions. Figure 3a displays the complete DAG of PyFlexTRKR, showing all tasks across stages and their data dependencies to illustrate the overall execution flow and inter-task relationships.

Overall, the workflow structure resulting from P-C relations is critical for proper interpretation, as it provides context for time, parallelism, data reuse, task type, and I/O type.

Finding 6: Producer-Consumer pattern-dependent I/O performance. Exchange patterns between P-C task groups, an inherent structural property of the workflow design, significantly influence I/O bandwidth efficiency. We observe that n - n patterns consistently achieve higher bandwidth than 1-1 patterns due to greater concurrency in data exchange. This performance advantage is reliable, with n - n patterns exhibiting low variability across runs (CV of only 9.6%). However, this advantage diminishes when dataflow sizes are very small (e.g., <20 MB). Figure 3b illustrates this, showing seven P-C groups achieving higher throughput through n - n or n -1 communication. A 1-1 exception, *idfea*→*trkst* (darker pink triangle), achieves similar high bandwidth due to its high concurrency (48 tasks on both sides). Understanding these P-C patterns is crucial for forecasting I/O performance under varying concurrency levels, enabling more informed data placement and effective storage resource utilization.

Finding 7: File-specific I/O diversity. In workflows, each group of files has its own unique I/O patterns. Figure 3d highlights how specific file groups contribute uniquely to the overall I/O workload. For instance, *cloudid*, shown in orange, spans a wide range of operation counts, while *mcstrack*, in grey, is tightly clustered around 3,000 and 9,500 operations. Both file types are frequently accessed in the workflow and exhibit a wide range of I/O bandwidth. Files like *mcs_tracks* in brown, *track* in green, and *trackstats* in purple have less frequent access, with all accesses occurring below 50 MB/s. Identifying such low-performance file accesses enables targeted optimization, such as migrating bottleneck files to faster storage tiers.

Finding 8: Dataflow lifetime variability. In workflows with many P-C connections that reflect a cascading access pattern, files are either short-lived (used and discarded within a

single stage) or long-lived (reused across multiple downstream tasks). Figure 3b shows that files like *cloudid* are reused across multiple stages. By combining this with the recorded I/O activity for the file type, we conclude that it has a longer dataflow lifetime within the workflow. These temporal variations in file access highlight opportunities to place hot data on faster storage tiers. When such long-lived files are also performance-limiting, the inefficiency is compounded throughout the entire workflow.

Finding 9: Temporal I/O hotspots. Different applications within a workflow often exhibit distinct I/O behaviors, resulting in varying I/O intensity over time. Figure 3f shows that I/O activity in the PyFlexTRKR workflow is not evenly distributed. Instead, certain stages, particularly stages 1, 8, and 9, experience concentrated bursts of I/O that form temporal hotspots. These peaks align with high-bandwidth P-C interactions. Performance improvement strategies for these hotspots include staging intermediate outputs on faster storage or scheduling tasks to reduce I/O contention.

Finding 10: I/O type performance difference. We focus on the key P-C pair *mapf*→*speed* (brown hexagon in Figure 3b) to demonstrate how I/O type governs performance in a high-concurrency exchange. Figure 3b shows this group has high dataflow but low proportional throughput, despite 48 tasks running on both sides. We examine the tasks in Figure 3c: the producer *mapf* (brown hexagon) has moderate bandwidth (22 to 277 MB/s), while the consumer *speed* (yellow triangle) has lower bandwidth (9 to 103 MB/s). Mapping to Figure 3e confirms the low-throughput clusters are composed of random writes (*mapf*) and random reads (*speed*). Finally, Figure 3d reveals that the consumer's random reads target the intermediate file *mcstracks*. This limited throughput is fundamentally due to *speed*'s low-bandwidth random reads on *mcstracks*, which are likely more sensitive to storage latency than the producer's writes.

D. Montage

Montage is a workflow for creating large-scale astronomical image mosaics, often used to generate Hierarchical Progressive Surveys maps from high-resolution survey data [26]. The workflow involves multiple I/O-intensive steps, including image reprojection, background matching, and coaddition, followed by hierarchical tiling. These tasks require reading and writing image files repeatedly, especially at high resolutions. Understanding the I/O patterns is critical for optimizing parallel execution, balancing data distribution, and avoiding storage bottlenecks.

1) Observation:

Figure 4 presents the I/O structure of the Montage workflow across multiple dimensions. Figure 4a illustrates the Montage workflow DAG, showing all tasks with cascading data dependencies. Task parallelism is depicted at a reduced scale and is not drawn to proportion. The workflow has an average task I/O size of 21 MB. Note that in the subfigures, a high-throughput point is omitted from the plot to improve the readability of the

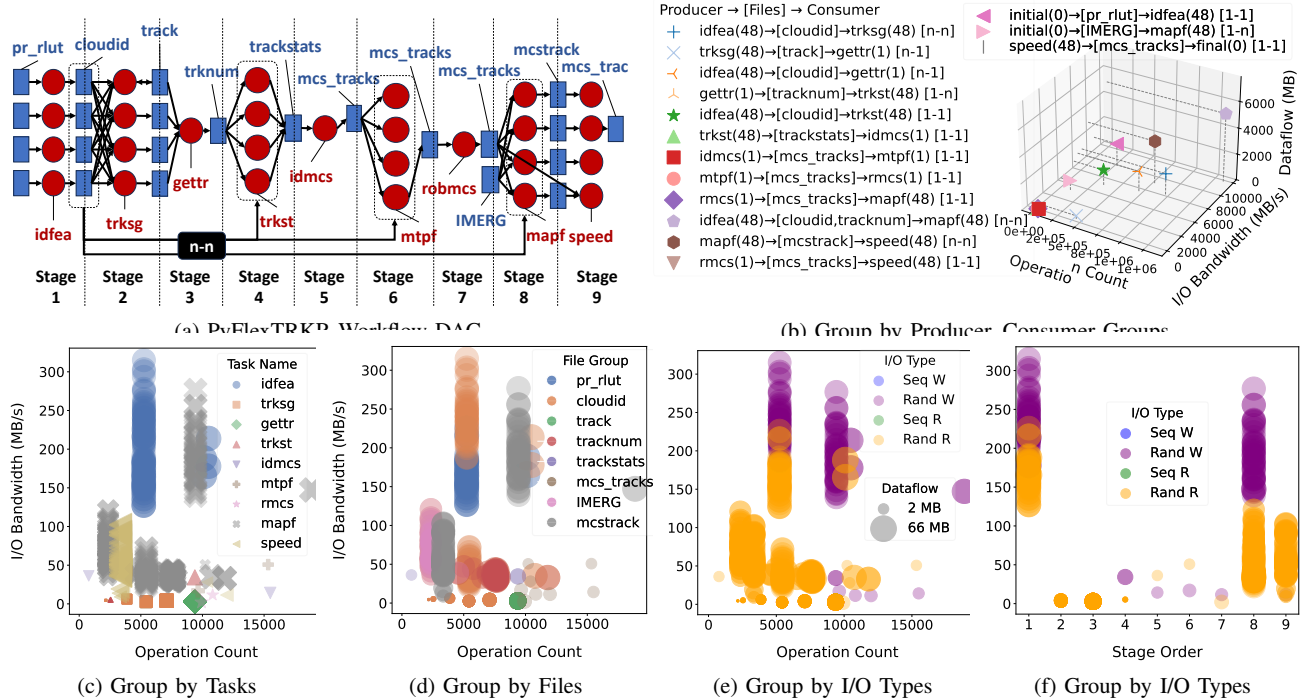


Fig. 3: Structural characterization and visualization of I/O profiles for the PyflexTRKR workflow.

remaining data points, facilitating analysis and highlighting bottlenecked tasks, which are the primary focus of this study.

Finding 11: P-C group analysis reveals I/O bottleneck. We focus on the *Pro*→*DiffF* P-C group (green star in Figure 4b) where the expected *n*-*n* benefit is contradicted by poor bandwidth. We examine the tasks in Figure 4e: the producer *Pro* (blue) operates in a low-bandwidth, low-operation-count region, while the consumer *DiffF* (orange) accesses higher-bandwidth zones and shows three distinct throughput clusters. Next, Figure 4d reveals that the low-performing activity involves a mix of sequential writes and sequential/random reads. Finally, Figure 4f links these low-performing operations to files like *region.hdr*, *pss2.fits*, and *1-diff.fits*. This suggests the inefficiency stems from small, fragmented sequential writes by *Pro* and poor read efficiency by *DiffF*, despite high parallelism. This pattern indicates that the file format or structure itself is a key contributor to the I/O bottleneck. This is a structural property of the workflow and its data formats, indicating that optimization would require changes to the workflow design or data format.

Finding 12: Intra-Stage Variation in I/O Patterns and Performance. Within a single workflow stage, different tasks can exhibit significantly different I/O patterns, leading to uneven demands on the storage system. Analysis of Figure 4g and Figure 4c shows that Stage 2 I/O operations are mostly clustered in the low-bandwidth range, with a few high-bandwidth outliers. Further analysis reveals that these high-performance outliers correspond to sequential write operations by task *DiffF* to the file *1-fit.txt*. In contrast, other files in Stage 2 show consistently low bandwidth.

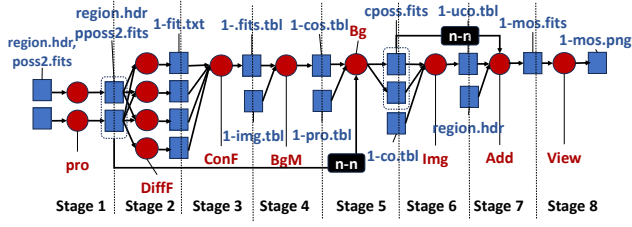
E. DDMD

DeepDriveMD (DDMD) is a deep learning-driven molecular dynamics simulation workflow for protein folding [27]. DDMD integrates simulation, ML training, and inference of new simulation parameters in a pipeline. Simulation and ML analysis integration is becoming prevalent in HPC workflows [28], [29], and DeepDriveMD exemplifies the I/O patterns of such workflows, which often exhibit diverse data access characteristics – a common feature in HPC simulation-analysis-visualization workflows [30], [31].

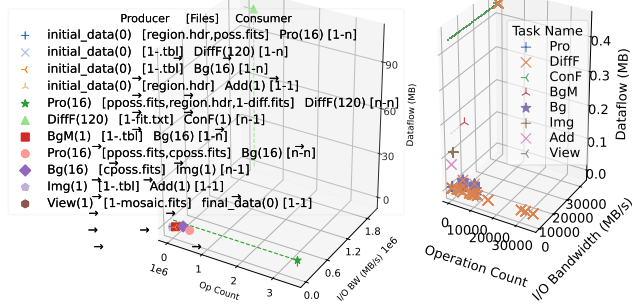
1) Observation:

Figure 5 presents the I/O behavior of the DDMD workflow, which follows an iterative structure composed of three main sequential stages: simulation (parallelized OpenMM tasks), aggregation, and AI-based analysis (training and inference). While simulation tasks operate in parallel, the downstream stages involve a single task per iteration. The workflow DAG (Figure 5a) reflects a 1-1 or 1-*n* P-C pattern across iterations, highlighting performance dynamics driven by repeated I/O interactions with shared files.

Finding 13: High-throughput tasks can mask workflow I/O inefficiencies. The final stage’s high throughput can create an illusion of efficiency, masking significant I/O constraints in upstream tasks. We first observe the workflow’s I/O behavior in Figure 5b: the *train*→*final* group (green triangle) achieves the highest throughput, but the *n*-1 groups, *sim*→*agg* (dark blue) and *sim*→*train* (light blue), exhibit much lower performance. We examine the tasks in Figure 5c: *sim* (blue) and *agg* (orange) are clustered in the low-bandwidth and low-dataflow region. In contrast, the *train* task (red) exhibits I/O across

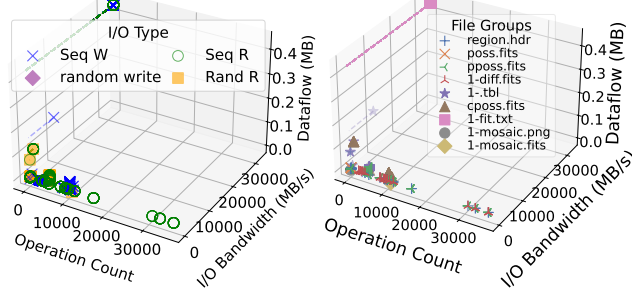


(a) Montage Workflow DAG



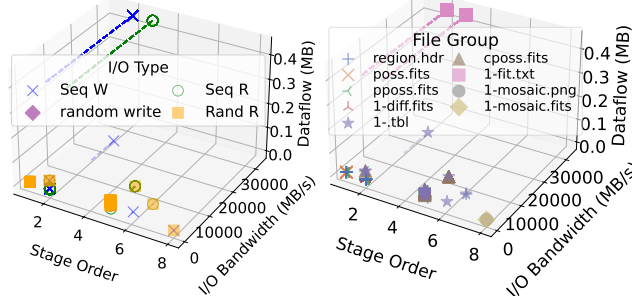
(b) Group by P-C Groups

(c) Group by Tasks



(d) Group by I/O Types

(e) Group by Files

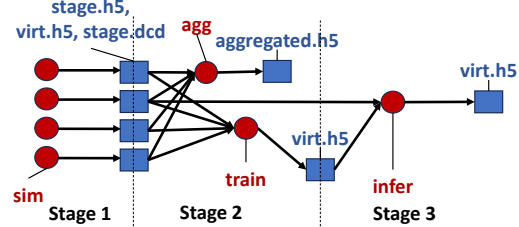


(f) Group by I/O Types

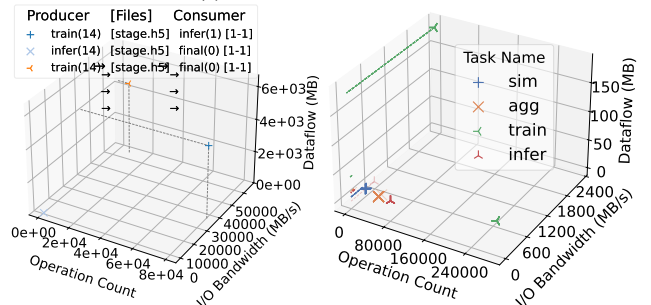
(g) Group by Files

Fig. 4: Structural characterization and visualization of I/O profiles for the Montage workflow.

two regions, driving the high-throughput illusion. Referring to Figure 5d and Figure 5e, the *train* task's high bandwidth comes from large dataflow write operations to checkpoint and embedding files (reaching 2.4 GB/s). Meanwhile, all I/O operations by *sim* and *agg* are concentrated in low-bandwidth, low-dataflow regions, confirming their limited throughput is driven by small dataflow sizes. The inference task is also heavily constrained, likely due to interactions with HDF5 files. The high bandwidth achieved by the *train*→*final* group

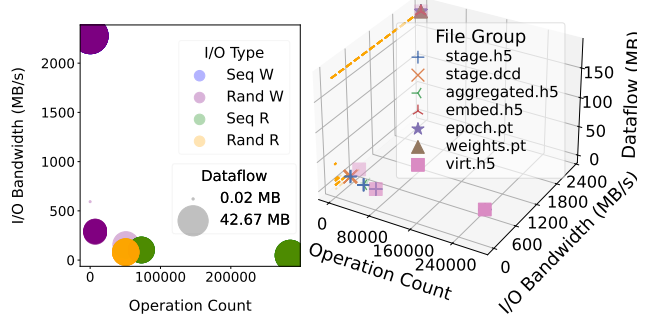


(a) DDMD Workflow DAG



(b) Group by P-C Groups

(c) Group by tasks



(d) Group by I/O types

(e) Group by files

Fig. 5: I/O characterization of the DeepDriveMD workflow. masks that other tasks are significantly more I/O-constrained. To improve end-to-end performance, upstream tasks with low dataflow size and poor bandwidth (like *sim* and *agg*) should be prioritized for batching or access restructuring.

Finding 14: Mismatches Between Data Format and Task I/O Patterns Degrade Performance. Using a structured data format like HDF5 does not guarantee high performance; efficiency is dictated by the application's access pattern. In DDMD, for instance, the *train* task achieves significantly higher throughput on HDF5 files than the *infer* task, which suggests its access pattern is better aligned with the file's internal layout (e.g., chunk size). Similar performance mismatches for HDF5 and FITS files were also observed in the PtychoNN and Montage workflows, respectively, demonstrating that task-specific access patterns (structural properties determined by how the workflow algorithms interact with data formats) are a primary driver of I/O efficiency.

F. PtychoNN

PtychoNN is a deep learning-based workflow for ptychographic imaging that replaces traditional iterative phase

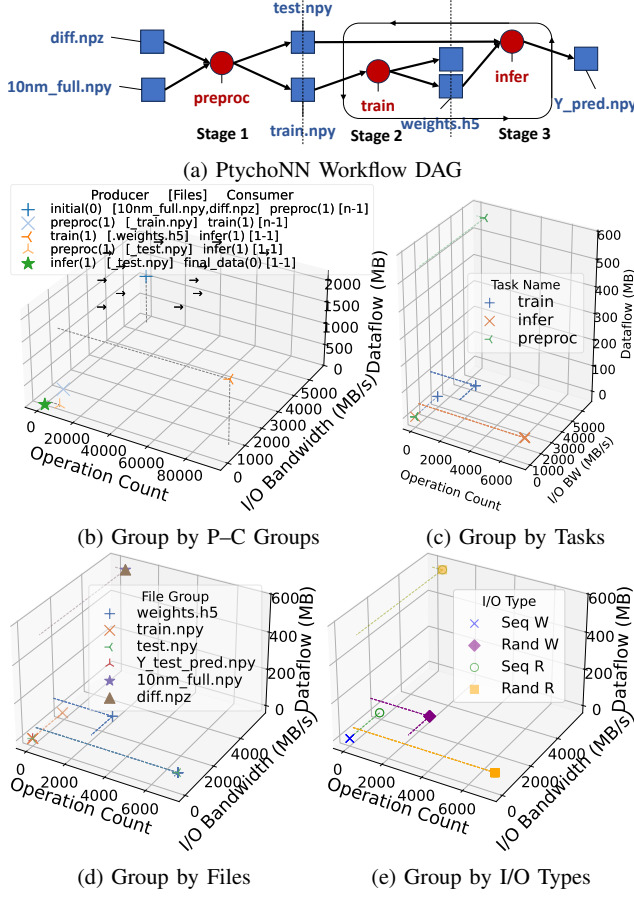


Fig. 6: I/O structure of the PtychoNN workflow across operation types, files, tasks, and producer–consumer interactions. retrieval with a fast, inference-driven approach [32]. The workflow supports high-throughput imaging of complex, dynamic, or dose-sensitive samples and is well-suited for deployment in HPC environments.

1) Observation:

Figure 6 presents the I/O structure of the PtychoNN workflow, which consists of three sequential stages: preprocessing, training, and inference. The workflow’s DAG is shown in Figure 6a. The preprocessing stage prepares training and testing data. Training and inference then run in a coordinated loop across epochs, with inference triggered by signals from training to read updated checkpoints. This creates frequent checkpoint reuse and a tightly coupled 1–1 P–C pattern.

Finding 15: Read–write performance asymmetry within tasks. Tasks can exhibit significant performance asymmetry between read and write operations, even for the same data format. Figure 6b shows that while the initial data reading group, *initial* → *preproc* (dark blue), achieves high bandwidth, other P–C groups experience significantly lower throughput. The *preproc* task (green in Figure 6c) exhibits the highest I/O performance, primarily from read operations shown in Figure 6e. This task reads compressed NumPy binary files, where lazy loading into memory yields higher

read bandwidth—contrary to typical cases where writes benefit from buffering and outperform reads.

Finding 16: Data preprocessing trade-offs. We observe that the *.npz* files exhibit worse read performance than *.npy* files. In this workflow, *.npz* files are used during preprocessing, where they are unzipped and split into training and testing datasets. Downstream tasks then operate primarily on *.npy* files, contributing to lower I/O throughput. The *.npy* files used in downstream stages show poorer read performance than their compressed *.npz* counterparts, suggesting a trade-off between preprocessing effort and long-term I/O efficiency. Employing techniques such as memory mapping, batching, or deferred decompression can help balance short-term preprocessing cost with long-term throughput benefits.

IV. CASE STUDIES

To demonstrate the practical impact of our characterization methodology, we present optimization case studies for three workflows where our analysis directly informed performance improvements. Table VI summarizes the findings, optimization strategies, experimental configurations, and achieved performance improvements.

a) 1000 Genomes Workflow. Findings 3 and 5 revealed caching effects and conflicting I/O patterns on shared files, indicating shared PFS contention as the primary bottleneck. As shown in Table VI, eliminating shared storage contention by migrating to node-local tmpfs achieved dramatic performance improvements. The additional gain from distributing workload across more nodes demonstrates that storage tier selection and task distribution work synergistically to mitigate I/O bottlenecks in workflows with high file reuse.

b) PyFlexTRKR Workflow. Findings 7 and 9 identified file-specific I/O diversity and temporal hotspots in Stages 1, 8, and 9. These insights motivated low-level analysis of Stage 9, which revealed that scattered small datasets were causing excessive metadata operations. We addressed this through data consolidation, merging multiple small datasets into larger, contiguous files while maintaining offset tracking. As detailed in Table VI, comparing consolidated versus unconsolidated layouts of the same data showed performance improvements most pronounced for smaller datasets and lower process counts, where metadata overhead dominated I/O time. This demonstrates how workflow-level characterization enables targeted optimization of metadata-intensive stages difficult to pinpoint using traditional system-level profiling.

c) DeepDriveMD Workflow. Findings 13 and 14 revealed that high-throughput downstream tasks can mask upstream bottlenecks and that access pattern misalignment with data format degrades performance. The train task’s high bandwidth masked poor throughput in sim and agg tasks, which suffered from misalignment with HDF5’s chunked layout. By switching to contiguous datasets as shown in Table VI, we aligned storage layout with workflow access patterns, directly addressing the masked bottleneck. This illustrates a critical advantage of workflow-level characterization: identifying bot-

Workflow	Key Findings	Optimization Strategy	Configurations Compared	Performance Gain
1000 Genomes	Finding 3: Caching effects (58× read speedup)	Storage tier selection	BeeGFS (PFS) vs. node-local SSD vs. node-local tmpfs	Up to 12.5× runtime improvement (PFS → tmpfs)
	Finding 5: Distinct I/O patterns on shared files		Scaling: 2, 5, 10 nodes (all on tmpfs)	3.4× improvement (2 → 10 nodes)
PyFlexTRKR	Finding 7: File-specific I/O diversity	Data layout consolidation	32 datasets, 23 accesses each; dataset sizes: 1KB–8KB	1.7× to 3.7× I/O time reduction
	Finding 9: Temporal hotspots (Stages 1, 8, 9)		Varying process counts on node-local NVMe	
DDMD	Finding 13: Downstream masking upstream bottlenecks	HDF5 layout tuning (chunked → contiguous)	Simulated sim/agg task I/O; varying dataset sizes and process counts	1.9× speedup in high-concurrency scenarios
	Finding 14: Access pattern misalignment with data format			

TABLE VI: Case study optimization results guided by workflow I/O characterization.

tlenecks obscured when examining only aggregate or final-stage performance metrics.

V. INSIGHTS AND LESSONS LEARNED

Many workflows exhibit significant variability in file reuse and access patterns. Some files are short-lived and stage-specific, while others persist across multiple stages, especially in iterative and cascading workflows. I/O activity is often temporally concentrated, with specific stages (e.g., scatter-gather, checkpointing) dominating data movement and forming temporal hotspots. Within stages, tasks may exhibit heterogeneous I/O behaviors, with files experiencing both small random reads and large sequential accesses. These patterns present clear opportunities: identifying hot or long-lived files enables staging to memory or fast storage [33], [34], while temporal hotspots and intra-task heterogeneity can guide task scheduling, caching, and bandwidth prioritization.

In analyzing P–C groups, 1–1 patterns (common in iterative or checkpointing workflows) often suffer from low bandwidth due to small dataflows and high operation counts, while n–n patterns (typical of scatter-gather workflows) achieve higher throughput via greater concurrency. These communication structures directly affect bandwidth and should inform optimization. Cascading workflows benefit from staging intermediate data in memory [35], while independent P–C groups can leverage parallelism or data duplication.

HPC workflows often use structured formats (e.g., HDF5, FITS, TAR), but I/O performance varies widely across tasks using the same format due to differences in access granularity, pattern, and concurrency—not format alone. This underscores the need for task- and format-aware optimizations and adaptive data placement aligned with access characteristics [20], [36].

Overall Conclusion: Converged workflows—integrating simulation, data analytics, AI training, and inference—exhibit diverse and interconnected dataflow patterns. These heterogeneous workflows require storage systems composed of distinct, concurrently accessible storage domains, each optimized for specific I/O characteristics such as throughput, latency, access size, and concurrency. Our findings underscore the need for future research in co-designing storage architectures and work-

flow schedulers that can dynamically align data movement patterns with appropriate storage tiers and access strategies.

Limitations and Future Work: Our study focuses on I/O activities visible at the user-space level, primarily through traceable system calls and application-level data access. As such, it does not capture direct memory-mapped accesses or low-level interactions that are handled internally by libraries or hardware, such as page caching, prefetching, or GPU direct memory access (DMA); tools exist for these analyses. Additionally, the static nature of post-mortem trace analysis limits our ability to capture transient runtime conditions, such as contention spikes, short-lived caching effects, or dynamic task migration.

Our *LD_PRELOAD* framework intercepted *mmap()* calls during tracing. We observed only a small number of memory-mapped I/O operations in DDMD, which had minimal impact on data and I/O flow. In other workflows including PtychoNN, we did not observe any memory-mapped I/O. For GPU Direct Memory Access, none of the studied workflows used GPU DMA. We note that comprehensive analysis of ML workflows would benefit from combining our workflow-level file I/O characterization with GPU-level tracing tools (such as Nvidia Nsight or DCGM) as important future work.

Our findings not only validate well-known I/O behaviors—such as performance penalties from small, fragmented accesses; they also uncover emerging patterns in modern ML and LLM workflows, such as heterogeneity in checkpointing behavior and reuse of large model parameter files. These observations emphasize the need for new task- and access-aware strategies, particularly in workflows that blend HPC simulations with AI components.

We plan to extend our workflow categorization and contextual analysis toward building an intelligent, I/O-aware provisioning service. This system would make data placement decisions based on both task–data dependencies and access pattern characteristics, enabling dynamic optimization across heterogeneous storage domains.

VI. RELATED WORK

a) HPC Workflow I/O analysis: As scientific applications produce increasingly large datasets, I/O bottlenecks remain

a major challenge in HPC workflows. Extensive research has sought to understand and optimize I/O behavior through profiling, modeling, and analysis. Tools such as Darshan [37], Recorder [38], IOPro [39], IOPin [40], and domain-specific profilers [41], [42] capture detailed I/O behavior and help identify performance bottlenecks. Benchmarking tools have also been used to characterize storage performance [43]. A suite of trace-based analysis tools builds on profiling data to uncover I/O inefficiencies [44], [45], [46], [47], [48], [49], [50], [51], [52], [53], [54]. Tools like DFTracer [55] specifically focus on I/O behavior and visualization in ML workloads. Other works explore access patterns [56], [57], [58], [59], [60], [61] and correlate application- and system-level I/O behavior [62], [63]. I/O modeling efforts examine storage characteristics [64], [65], [66], runtime traces [67], [68], [69], and server-side logs [51] to inform scheduling decisions. Studies of HPC I/O workloads highlight diverse patterns and I/O inefficiencies [70], [12], [71], [72], [73], [74], [75], [76], [77], [78], [79], [80]. Additional research includes automated workload characterization [81] and production system monitoring [62]. However, most prior work focuses on storage or application-level performance without capturing the interactions between tasks and data within the context of producer–consumer relationships in workflow execution. Recent studies, such as DataFlow Lifecycles [20], [82], investigate workflow-level I/O analysis and I/O-aware scheduling, while storage configuration frameworks [83] further enhance workflow coordination. DaYu [36] additionally analyzes I/O semantics in relation to workflow dataflows, and workflow-level analysis using logs [84] provides further insights. Large-scale workload surveys reveal trends in I/O behavior and help guide the design of future systems [85]. All these efforts emphasize the importance of identifying data dependencies to improve data placement decisions. However, they still lack comprehensive analysis across diverse workflows and scientific domains and do not provide deeper insights into data usage patterns that could inform storage resource selection.

b) I/O-Aware Scheduling: Traditional HPC workflow schedulers or task orchestration systems focus on computation and often lack consideration of I/O. Many existing compute- and task-based workflow scheduling approaches [18], [86], [87], [88], [89], [90] improve execution without considering storage resource performance. Coupled workflows reduce data movement [91], [92], [93], [94], [95] but struggle with complex producer–consumer patterns. Others have explored DAG-based scheduling methods [96], [97], [98], using graph partitioning strategies like min-flow/max-cut [99] and multilevel heuristics [100], [15] to reduce communication costs. Tools like METIS [98], [97] and dagP [96] optimize throughput, while HEFT and CPOP [101] and their extensions [102], [103], [104], [16] prioritize response time using critical paths. Runtime-aware [105] and dataflow-informed [106] methods further refine these decisions. However, most DAG-based techniques remain focused on compute or communication metrics, lacking I/O-aware coordination. Data-centric approaches [107], [108], [109], [110], [111], [112] address data

locality, with DFMan assigning tasks to storage resources to optimize bandwidth. Yet, it still prioritizes bandwidth over response time and lacks global modeling of dataflows. Recent I/O-aware schedulers [113], [114] improve storage utilization but lack workflow-level insight and the ability to model producer–consumer dynamics for storage optimization.

VII. CONCLUSION

While prior studies have examined I/O behavior in scientific applications, they often rely on system-level metrics or synthetic benchmarks, offering limited insight into how workflow structure influences I/O performance. This work presents the first large-scale, workflow-centric I/O analysis capturing fine-grained task–file behavior across diverse domains—including genomics, climate science, seismology, molecular dynamics, and machine learning. We introduce a methodology that correlates workflow context—such as task composition, execution stages, and producer–consumer dependencies—with detailed I/O metrics like access type, operation count, and dataflow size, enabling classification of common producer–consumer patterns and revealing workflow-specific I/O dynamics. Through holistic evaluation of six real-world workflows, we observe that 1–1 producer–consumer patterns often underutilize bandwidth, while n – n structures benefit from higher concurrency. Temporal I/O hotspots and I/O heterogeneity—both across tasks and within files—highlight optimization opportunities. File reuse and format-dependent access patterns also significantly influence performance and data placement. These findings expose inefficiencies missed by traditional profiling and point to new optimization directions. By linking workflow design with I/O behavior, our work enables I/O-aware scheduling and storage coordination, allowing systems to dynamically align task execution and data movement with access patterns to improve workflow performance at scale.

ACKNOWLEDGMENTS

This research is partly supported by the National Science Foundation under Grants No. CSSI-2104013 and OAC-2313154, and partly by the U.S. Department of Energy (DOE) through the Office of Advanced Scientific Computing Research’s “Orchestration for Distributed & Data-Intensive Scientific Exploration” and the “Decentralized data mesh for autonomous materials synthesis” AT SCALE LDRD at Pacific Northwest National Laboratory. PNNL is operated by Battelle for the DOE under Contract DE-AC05-76RL01830.

REFERENCES

- [1] K. Mehta, A. Cliff, F. Suter, A. M. Walker, M. Wolf, D. Jacobson, and S. Klasky, “Running ensemble workflows at extreme scale: Lessons learned and path forward,” in *2022 IEEE 18th Intl. Conf. on e-Science (e-Science)*, 2022, pp. 284–294.
- [2] A. Brace, S. Jha, I. Yakushin, H. Lee, H. Ma, A. Trifan, L. Tan, T. Munson, M. Turilli, I. Foster, and A. Ramanathan, “Coupling streaming ai and hpc ensembles to achieve 100-1000× faster biomolecular simulations,” in *2022 IEEE Intl. Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2022.

- [3] A. Merzky, M. Turilli, M. Titov, A. Al-Saadi, and S. Jha, "Design and performance characterization of radical-pilot on leadership-class platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 818–829, 2022.
- [4] S. V. Kalinin, E. Strelcov, A. Belianinov, S. Somnath, R. K. Vasudevan, E. J. Lingerfelt, R. K. Archibald, C. Chen, R. Proksch, N. Laanait *et al.*, "Big, deep, and smart data in scanning probe microscopy," 2016.
- [5] B. Allcock, I. Foster, V. Nefedova, A. Chervenak, E. Deelman, C. Kesselman, J. Lee, A. Sim, A. Shoshani, B. Drach *et al.*, "High-performance remote access to climate simulation data: A challenge problem for data grid technologies," in *Proceedings of the 2001 ACM/IEEE conference on Supercomputing*, 2001, pp. 46–46.
- [6] M. Chi, A. Plaza, J. A. Benediktsson, Z. Sun, J. Shen, and Y. Zhu, "Big data for remote sensing: Challenges and opportunities," *Proceedings of the IEEE*, vol. 104, no. 11, pp. 2207–2219, 2016.
- [7] E. Deelman, T. Peterka, I. Altintas, C. D. Carothers, K. K. Van Dam, K. Moreland, M. Parashar, L. Ramakrishnan, M. Taufer, and J. Vetter, "The future of scientific workflows," *The International Journal of High Performance Computing Applications*, vol. 32, no. 1, pp. 159–175, 2018.
- [8] J. Yeom and D. Ahn, "Ubique: A new data object transfer model for hpc workflows," Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), Tech. Rep., 2021.
- [9] H. Bhatia, F. Di Natale, J. Y. Moon, X. Zhang, J. R. Chavez, F. Aydin, C. Stanley, T. Oppelstrup, C. Neale, S. K. Schumacher *et al.*, "Generalizable coordination of large multiscale workflows: challenges and learnings at scale," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–16.
- [10] J. Dongarra, P. Beckman, T. Moore, P. Aerts, G. Aloisio, J.-C. Andre, D. Barkai, J.-Y. Berthou, T. Boku, B. Braunschweig *et al.*, "The international exascale software project roadmap," *The international journal of high performance computing applications*, vol. 25, no. 1, pp. 3–60, 2011.
- [11] O. Yildiz, M. Dorier, S. Ibrahim, R. Ross, and G. Antoniu, "On the root causes of cross-application i/o interference in hpc storage systems," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2016, pp. 750–759.
- [12] J. L. Bez, S. Byna, and S. Ibrahim, "I/o access patterns in hpc applications: A 360-degree survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–41, 2023.
- [13] A. Kougkas, H. Devarajan, J. Lofstead, and X.-H. Sun, "Labios: A distributed label-based i/o system," in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, 2019, pp. 13–24.
- [14] L. Logan, J. C. Garcia, J. Lofstead, X.-H. Sun, and A. Kougkas, "Labstor: A modular and extensible platform for developing high-performance, customized i/o stacks in userspace," in *Proceedings of the Intl. Conf. on High Performance Computing, Networking, Storage and Analysis*, ser. SC '22. IEEE Press, 2022.
- [15] M. Y. Özkaya, A. Benoit, and Ü. V. Çatalyürek, "Improving locality-aware scheduling with acyclic directed graph partitioning," in *Parallel Processing and Applied Mathematics*, R. Wyrzykowski, E. Deelman, J. Dongarra, and K. Karczewski, Eds. Cham: Springer Intl. Publishing, 2020, pp. 211–223.
- [16] B. Wu, Z. Xiao, P. Lin, Z. Tang, and K. Li, "Critical path awareness techniques for large-scale graph partitioning," *IEEE Transactions on Sustainable Computing*, vol. 8, no. 3, pp. 412–422, 2023.
- [17] M. R. Wyatt, S. Herbein, T. Gamblin, and M. Taufer, "Ai4io: A suite of ai-based tools for io-aware scheduling," *The International Journal of High Performance Computing Applications*, vol. 36, no. 3, pp. 370–387, 2022.
- [18] E. Deelman, K. Vahi, G. Juve, M. Rynge, S. Callaghan, P. J. Maechling, R. Mayani, W. Chen, R. F. Da Silva, M. Livny *et al.*, "Pegasus, a workflow management system for science automation," *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015.
- [19] W. P. Computing and I. Foster, "Designing and building parallel programs," 1995.
- [20] H. Lee, L. Guo, M. Tang, J. Firoz, N. Tallent, A. Kougkas, and X.-H. Sun, "Data flow lifecycles for optimizing workflow coordination," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3581784.3607104>
- [21] H. Lee, J. Firoz, N. R. Tallent, L. Guo, and M. Halappanavar, "Flow-forecaster: Automatically inferring detailed & interpretable workflow scaling models for forecasts," in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2025, pp. 420–432.
- [22] K. Mehta, A. Cliff, F. Suter, A. M. Walker, M. Wolf, D. Jacobson, and S. Klasky, "Running ensemble workflows at extreme scale: Lessons learned and path forward," in *2022 IEEE 18th International Conference on e-Science (e-Science)*, 2022, pp. 284–294.
- [23] "Seismic cross correlation workflow git repo," <https://github.com/wfcommons/pegasus-instances/tree/master/seismology>, accessed: 2023-03-15.
- [24] L. Clarke, X. Zheng-Bradley, R. Smith, E. Kulesha, C. Xiao, I. Toneva, B. Vaughan, D. Preuss, R. Leinonen, M. Shumway *et al.*, "The 1000 genomes project: data management and community access," *Nature methods*, 2012.
- [25] Z. Feng, J. Hardin, H. C. Barnes, J. Li, L. R. Leung, A. Varble, and Z. Zhang, "Pyflextrkr: a flexible feature tracking python software for convective cloud analysis," *EGU sphere*, pp. 1–29, 2022.
- [26] M. Rynge, G. Juve, J. Kinney, J. Good, B. Berriman, A. Merrihew, and E. Deelman, "Producing an infrared multiwavelength galactic plane atlas using montage, pegasus and amazon web services," in *23rd Annual Astronomical Data Analysis Software and Systems, ADASS, Conference*, 2013.
- [27] H. Lee, M. Turilli, S. Jha, D. Bhowmik, H. Ma, and A. Ramanathan, "Deepdrivemd: Deep-learning driven adaptive molecular simulations for protein folding," in *2019 IEEE/ACM Third Workshop on Deep Learning on Supercomputers (DLS)*. IEEE, 2019, pp. 12–19.
- [28] J. Wei, X. Chu, X.-Y. Sun, K. Xu, H.-X. Deng, J. Chen, Z. Wei, and M. Lei, "Machine learning in materials science," *InfoMat*, vol. 1, no. 3, pp. 338–358, 2019.
- [29] P. A. O'Gorman and J. G. Dwyer, "Using machine learning to parameterize moist convection: Potential for modeling of climate, climate change, and extreme events," *Journal of Advances in Modeling Earth Systems*, vol. 10, no. 10, pp. 2548–2563, 2018.
- [30] C. Peña-Monferrer, R. Manson-Sawko, and V. Elisseev, "Hpc-cloud native framework for concurrent simulation, analysis and visualization of cfd workflows," *Future Generation Computer Systems*, vol. 123, pp. 14–23, 2021.
- [31] J. Senk, A. Yegenoglu, O. Amblet, Y. Brukau, A. Davison, D. R. Lester, A. Lührs, P. Quaglio, V. Rostami, A. Rowley *et al.*, "A collaborative simulation-analysis workflow for computational neuroscience using hpc," in *High-Performance Scientific Computing: First JARA-HPC Symposium, JHPCS 2016, Aachen, Germany, October 4–5, 2016, Revised Selected Papers 1*. Springer, 2017, pp. 243–256.
- [32] M. J. Cherukara, T. Zhou, Y. Nashed, P. Enfedaque, A. Hexemer, R. J. Harder, and M. V. Holt, "Ai-enabled high-resolution scanning coherent diffraction imaging," *Applied Physics Letters*, vol. 117, no. 4, 2020.
- [33] A. Kougkas, H. Devarajan, and X.-H. Sun, "Hermes: A heterogeneous-aware multi-tiered distributed i/o buffering system," in *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 219–230. [Online]. Available: <https://doi.org/10.1145/3208040.3208059>
- [34] L. Pottier, R. F. da Silva, H. Casanova, and E. Deelman, "Modeling the performance of scientific workflow executions on hpc platforms with burst buffers," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2020, pp. 92–103.
- [35] M. Larsen, J. Ahrens, U. Ayachit, E. Brugger, H. Childs, B. Geveci, and C. Harrison, "The alpine in situ infrastructure: Ascending from the ashes of strawman," in *Proce. of the In Situ Infrastructures on Enabling Extreme-Scale Analysis and Visualization*. New York, NY, USA: Association for Computing Machinery, 2017, pp. 42–46.
- [36] M. Tang, J. Cernuda, J. Ye, L. Guo, N. R. Tallent, A. Kougkas, and X.-H. Sun, "Dayu: Optimizing distributed scientific workflows by decoding dataflow semantics and dynamics," in *Proceedings of the 2024 IEEE International Conference on Cluster Computing (CLUSTER)*, ser. CLUSTER '24, IEEE. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers, September 2024, pp. 357–369.
- [37] P. Carns, K. Harms, W. Allcock, C. Bacon, S. Lang, R. Latham, and R. Ross, "Understanding and improving computational science storage access through continuous characterization," *ACM Transactions on Storage (TOS)*, vol. 7, no. 3, pp. 1–26, 2011.

- [38] C. Wang, J. Sun, M. Snir, K. Mohror, and E. Gonsiorowski, "Recorder 2.0: Efficient parallel i/o tracing and analysis," in *2020 IEEE Intl. Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2020, pp. 1–8.
- [39] J. Schindler, "Profiling and analyzing the i/o performance of nosql dbs," in *Proceedings of the ACM SIGMETRICS/international conference on Measurement and modeling of computer systems*, 2013, pp. 389–390.
- [40] S. J. Kim, S. W. Son, W.-k. Liao, M. Kandemir, R. Thakur, and A. Choudhary, "Iopin: Runtime profiling of parallel i/o in hpc systems," in *2012 SC Companion: High Performance Computing, Networking Storage and Analysis*. IEEE, 2012, pp. 18–23.
- [41] Z. Liu, B. Wang, T. Wang, Y. Tian, C. Xu, Y. Wang, W. Yu, C. A. Cruz, S. Zhou, T. Clune *et al.*, "Profiling and improving i/o performance of a large-scale climate scientific application," in *2013 22nd International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2013, pp. 1–7.
- [42] S. J. Kim, "Parallel i/o profiling and optimization in hpc systems," 2014.
- [43] H. Shan and J. Shalf, "Using ior to analyze the i/o performance for hpc platforms," 2007.
- [44] G. K. Lockwood, N. J. Wright, S. Snyder, P. Carns, G. Brown, and K. Harms, "Tokio on clusterstor: connecting standard tools to enable holistic i/o performance analysis," Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States), Tech. Rep., 2018.
- [45] T. Wang, S. Snyder, G. Lockwood, P. Carns, N. Wright, and S. Byna, "Iominer: Large-scale analytics framework for gaining knowledge from i/o logs," in *2018 IEEE Intl. Conf. on Cluster Computing (CLUSTER)*. IEEE, 2018, pp. 466–476.
- [46] I. Yildirim, H. Devarajan, A. Kougkas, X.-H. Sun, and K. Mohror, "Iomax: Maximizing out-of-core i/o analysis performance on hpc systems," in *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1209–1215. [Online]. Available: <https://doi.org/10.1145/3624062.3624191>
- [47] K. Ramakrishnan, P. Biswas, and R. Karedla, "Analysis of file i/o traces in commercial computing environments," *ACM SIGMETRICS Performance Evaluation Review*, vol. 20, no. 1, pp. 78–90, 1992.
- [48] J. L. Bez, H. Ather, and S. Byna, "Drishti: Guiding end-users in the i/o optimization journey," in *2022 IEEE/ACM International Parallel Data Systems Workshop (PDSW)*. IEEE, 2022, pp. 1–6.
- [49] N. Nakka, A. Choudhary, W. Liao, L. Ward, R. Klundt, and M. I. Weston, "Detailed analysis of i/o traces for large scale applications," in *2009 International Conference on High Performance Computing (HiPC)*. IEEE, 2009, pp. 419–427.
- [50] S. A. Wright, S. D. Hammond, S. J. Pennycook, R. F. Bird, J. Herdman, I. Miller, A. Vadgama, A. Bhalerao, and S. A. Jarvis, "Parallel file system analysis through application i/o tracing," *The Computer Journal*, vol. 56, no. 2, pp. 141–155, 2013.
- [51] Y. Liu, R. Gunasekaran, X. Ma, and S. S. Vazhkudai, "Server-side log data analytics for i/o workload characterization and coordination on large shared storage systems," in *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 819–829.
- [52] P. Lu and K. Shen, "Multi-layer event trace analysis for parallel i/o performance tuning," in *2007 International Conference on Parallel Processing (ICPP 2007)*. IEEE, 2007, pp. 12–12.
- [53] A. Konwinski, J. Bent, J. Nunez, and M. Quist, "Towards an i/o tracing framework taxonomy," in *Proceedings of the 2nd international workshop on Petascale data storage: held in conjunction with Supercomputing'07*, 2007, pp. 56–62.
- [54] G. Malecha, G. Morrisett, and R. Wisnesky, "Trace-based verification of imperative programs with i/o," *Journal of Symbolic Computation*, vol. 46, no. 2, pp. 95–118, 2011.
- [55] H. Devarajan, L. Pottier, K. Velusamy, H. Zheng, I. Yildirim, O. Kogio, W. Yu, A. Kougkas, X.-H. Sun, J. S. Yeom *et al.*, "Dfracer: An analysis-friendly data flow tracer for ai-driven workflows," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–24.
- [56] T. Patel and S. Byna, "Uncovering access, reuse, and sharing characteristics of i/o-intensive files on large-scale production hpc systems," in *Proceedings of the 18th USENIX Conf. on File and Storage Technologies*, 2020, 2020.
- [57] T. Patel, S. Byna, G. K. Lockwood, and D. Tiwari, "Revisiting i/o behavior in large-scale storage systems: The expected and the unexpected," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–13.
- [58] E. Strohmaier and H. Shan, "Apex-map: A global data access benchmark to analyze hpc systems and parallel programming paradigms," in *SC'05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*. IEEE, 2005, pp. 49–49.
- [59] G. K. Lockwood, S. Snyder, S. Byna, P. Carns, and N. J. Wright, "Understanding data motion in the modern hpc data center," in *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, 2019, pp. 74–83.
- [60] J. L. Bez, F. Z. Boito, R. Nou, A. Miranda, T. Cortes, and P. O. Navaux, "Detecting i/o access patterns of hpc workloads at runtime," in *2019 31st International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 2019, pp. 80–87.
- [61] S. Sehrish, G. Mackey, J. Wang, and J. Bent, "Mrap: a novel mapreduce-based framework to support hpc analytics applications with access patterns," in *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, ser. HPDC '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 107–118. [Online]. Available: <https://doi.org/10.1145/1851476.1851490>
- [62] L. Wan, M. Wolf, F. Wang, J. Y. Choi, G. Ostrouchov, and S. Klasky, "Comprehensive measurement and analysis of the user-perceived i/o performance in a production leadership-class storage system," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 1022–1031.
- [63] S. Madireddy, P. Balaprakash, P. Carns, R. Latham, R. Ross, S. Snyder, and S. M. Wild, "Analysis and correlation of application i/o performance and system-wide i/o activity," in *2017 International Conference on Networking, Architecture, and Storage (NAS)*, 2017, pp. 1–10.
- [64] Y. Kim and R. Gunasekaran, "Understanding i/o workload characteristics of a peta-scale storage system," *The Journal of Supercomputing*, vol. 71, pp. 761–780, 2015.
- [65] T. I. Papon and M. Athanassoulis, "A parametric i/o model for modern storage devices," in *Proceedings of the 17th International Workshop on Data Management on New Hardware*, 2021, pp. 1–11.
- [66] Y. He, D. Dai, and F. S. Bao, "Modeling hpc storage performance using long short-term memory networks," in *2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 2019, pp. 1107–1114.
- [67] S. Byna, X.-H. Sun, and D. Holmgren, "Modeling data access contention in multicore architectures," in *2009 15th International Conference on Parallel and Distributed Systems*. IEEE, 2009, pp. 213–219.
- [68] Q. Noorshams, K. Rostami, S. Kounev, P. Tuma, and R. Reussner, "I/o performance modeling of virtualized storage systems," in *2013 IEEE 21st International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems*. IEEE, 2013, pp. 121–130.
- [69] Z. Ren and A. Trivedi, "Performance characterization of modern storage stacks: Posix i/o, libaio, spdck, and io_uring," in *Proceedings of the 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, 2023, pp. 35–45.
- [70] S. Neuwirth and A. K. Paul, "Parallel i/o evaluation techniques and emerging hpc workloads: A perspective," in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2021, pp. 671–679.
- [71] A. K. Paul, A. M. Karimi, and F. Wang, "Characterizing machine learning i/o workloads on leadership scale hpc systems," in *2021 29th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2021, pp. 1–8.
- [72] S. Snyder, P. Carns, R. Latham, M. Mubarak, R. Ross, C. Carothers, B. Behzad, H. V. T. Luu, S. Byna, and Prabhat, "Techniques for modeling large-scale hpc i/o workloads," in *Proceedings of the 6th International Workshop on Performance Modeling, Benchmarking, and Simulation of High Performance Computing Systems*, 2015, pp. 1–11.
- [73] A. K. Paul, A. M. Karimi, and F. Wang, "Characterizing machine learning i/o workloads on leadership scale hpc systems," in *2021 29th International Symposium on Modeling, Analysis, and Simulation of*

- Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2021, pp. 1–8.
- [74] A. Bandet, “Characterization and monitoring of i/o for hpc workloads,” Ph.D. dissertation, Université de Bordeaux, 2024.
 - [75] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, “Characterizing and profiling scientific workflows,” *Future generation computer systems*, vol. 29, no. 3, pp. 682–692, 2013.
 - [76] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M.-H. Su, and K. Vahi, “Characterization of scientific workflows,” in *2008 third workshop on workflows in support of large-scale science*. IEEE, 2008, pp. 1–10.
 - [77] P. Krawczuk, G. Papadimitriou, R. Tanaka, T. M. A. Do, S. Subramanya, S. Nagarkar, A. Jain, K. Lam, A. Mandal, L. Pottier *et al.*, “A performance characterization of scientific machine learning workflows,” in *2021 IEEE Workshop on Workflows in Support of Large-Scale Science (WORKS)*. IEEE, 2021, pp. 58–65.
 - [78] J. P. White, A. D. Kofke, R. L. DeLeon, M. Innus, M. D. Jones, and T. R. Furlani, “Automatic characterization of hpc job parallel filesystem i/o patterns,” in *Proceedings of the Practice and Experience on Advanced Research Computing: Seamless Creativity*, 2018, pp. 1–8.
 - [79] O. Kogiou, H. Devarajan, C. Wang, W. Yu, and K. Mohror, “I/o characterization of heterogeneous workflows.”
 - [80] H. Devarajan and K. Mohror, “Extracting and characterizing i/o behavior of hpc workloads,” in *2022 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2022, pp. 243–255.
 - [81] Z. Zhu, S. Neuirth, and T. Lippert, “A comprehensive i/o knowledge cycle for modular and automated hpc workload analysis,” in *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, 2022, pp. 581–588.
 - [82] L. Guo, H. Lee, J. Firoz, M. Tang, and N. R. Tallent, “Improving I/O-aware workflow scheduling via data flow characterization and trade-off analysis,” in *Seventh IEEE Intl. Workshop on Benchmarking, Performance Tuning and Optimization for Big Data Applications (Proc. of the IEEE Intl. Conf. on Big Data)*. IEEE Computer Society, December 2024.
 - [83] L. B. Costa, S. Al-Kiswani, H. Yang, and M. Ripeanu, “Supporting storage configuration for i/o intensive workflows,” in *Proceedings of the 28th ACM international conference on Supercomputing*, 2014, pp. 191–200.
 - [84] D. Ghoshal, B. Austin, D. Bard, C. Daley, G. Lockwood, N. J. Wright, and L. Ramakrishnan, “Characterizing scientific workflows on hpc systems using logs,” in *2020 IEEE/ACM Workflows in Support of Large-Scale Science (WORKS)*, 2020, pp. 57–64.
 - [85] P. Carns, K. Harms, W. Allcock, C. Bacon, S. Lang, R. Latham, and R. Ross, “Understanding and improving computational science storage access through continuous characterization,” *ACM Trans. Storage*, vol. 7, no. 3, Oct. 2011. [Online]. Available: <https://doi.org/10.1145/2027066.2027068>
 - [86] H.-J. Jiang, K.-C. Huang, H.-Y. Chang, D.-S. Gu, and P.-J. Shih, “Scheduling concurrent workflows in hpc cloud through exploiting schedule gaps,” in *Algorithms and Architectures for Parallel Processing: 11th International Conference, ICA3PP, Melbourne, Australia, October 24-26, 2011, Proceedings, Part I 11*. Springer, 2011, pp. 282–293.
 - [87] T. S. Somasundaram and K. Govindarajan, “Cloudrb: A framework for scheduling and managing high-performance computing (hpc) applications in science cloud,” *Future Generation Computer Systems*, vol. 34, pp. 47–65, 2014.
 - [88] R. S. Barga, D. Fay, D. Guo, S. Newhouse, Y. Simmhan, and A. Szalay, “Efficient scheduling of scientific workflows in a high performance computing cluster,” in *Proceedings of the 6th international workshop on Challenges of large applications in distributed environments*, 2008, pp. 63–68.
 - [89] F. Dong and S. G. Akl, “Scheduling algorithms for grid computing: State of the art and open problems,” *School of Computing, Queen’s University, Kingston, Ontario*, pp. 1–55, 2006.
 - [90] K. L. Krause, V. Y. Shen, and H. D. Schwetman, “Analysis of several task-scheduling algorithms for a model of multiprogramming computer systems,” *Journal of the ACM (JACM)*, vol. 22, no. 4, pp. 522–550, 1975.
 - [91] P. Subedi, P. Davis, S. Duan, S. Klasky, H. Kolla, and M. Parashar, “Stacker: an autonomic data movement engine for extreme-scale data staging-based in-situ workflows,” in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2018, pp. 920–930.
 - [92] S. Jin, S. Di, F. Vivien, D. Wang, Y. Robert, D. Tao, and F. Cappello, “Concealing compression-accelerated i/o for hpc applications through in situ task scheduling,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 981–998.
 - [93] F. Zhang, C. Docan, M. Parashar, S. Klasky, N. Podhorszki, and H. Abbasi, “Enabling in-situ execution of coupled scientific workflow on multi-core platform,” in *2012 IEEE 26th International Parallel and Distributed Processing Symposium*. IEEE, 2012, pp. 1352–1363.
 - [94] C. Docan, M. Parashar, and S. Klasky, “Dataspace: an interaction and coordination framework for coupled simulation workflows,” in *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, 2010, pp. 25–36.
 - [95] P. E. Davis, P. Subedi, S. Duan, L. Ricketson, J. A. Hittinger, and M. Parashar, “Benesh: a programming model for coupled scientific workflows,” in *2020 IEEE/ACM Fifth International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*. IEEE, 2020, pp. 1–9.
 - [96] J. Hermann, M. Y. Özkaya, B. Uçar, K. Kaya, and U. V. Çatalyürek, “Multilevel algorithms for acyclic partitioning of directed acyclic graphs,” *SIAM Journal on Scientific Computing*, vol. 41, no. 4, pp. A2117–A2145, 2019.
 - [97] G. Karypis and V. Kumar, “A fast and high quality multilevel scheme for partitioning irregular graphs,” *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998. [Online]. Available: <https://doi.org/10.1137/S1064827595287997>
 - [98] —, “Analysis of multilevel graph partitioning,” in *Proceedings of the 1995 ACM/IEEE Conf. on Supercomputing*, ser. Supercomputing ’95. New York, NY, USA: Association for Computing Machinery, 1995, pp. 29–es. [Online]. Available: <https://doi.org/10.1145/224170.224229>
 - [99] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
 - [100] B. Hendrickson, R. W. Leland *et al.*, “A multi-level algorithm for partitioning graphs,” *SC*, vol. 95, no. 28, pp. 1–14, 1995.
 - [101] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
 - [102] R. Mayer, C. Mayer, and L. Laich, “The tensorflow partitioning and scheduling problem: It’s the critical path!” in *Proceedings of the 1st Workshop on Distributed Infrastructures for Deep Learning*, ser. DIDL ’17. New York, NY, USA: Association for Computing Machinery, 2017, pp. 1–6. [Online]. Available: <https://doi.org/10.1145/3154842.3154843>
 - [103] K. Almi’ani and Y. C. Lee, “Partitioning-based workflow scheduling in clouds,” in *2016 IEEE 30th Intl. Conf. on Advanced Information Networking and Applications (AINA)*, 2016, pp. 645–652.
 - [104] G. Buttazzo, E. Bini, and Y. Wu, “Partitioning real-time applications over multicore reservations,” *IEEE Transactions on Industrial Informatics*, vol. 7, no. 2, pp. 302–315, 2011.
 - [105] D. H. Ahn, X. Zhang, J. Mast, S. Herbein, F. Di Natale, D. Kirshner, S. A. Jacobs, I. Karlin, D. J. Milroy, B. De Supinski, B. Van Essen, J. Allen, and F. C. Lightstone, “Scalable composition and analysis techniques for massive scientific workflows,” in *2022 IEEE 18th Intl. Conf. on e-Science (e-Science)*, 2022, pp. 32–43.
 - [106] H. Lee, L. Guo, M. Tang, J. Firoz, N. Tallent, A. Kougkas, and X.-H. Sun, “Data flow lifecycles for optimizing workflow coordination,” in *Proc. of the Intl. Conf. for High Performance Computing, Networking, Storage and Analysis (SuperComputing)*, ser. SC ’23. New York, NY, USA: Association for Computing Machinery, November 2023. [Online]. Available: <https://doi.org/10.1145/3581784.3607104>
 - [107] F. Chowdhury, F. Di Natale, A. Moody, K. Mohror, and W. Yu, “DFMan: A graph-based optimization of dataflow scheduling on high-performance computing systems,” in *2022 IEEE Intl. Parallel and Distributed Processing Symposium (IPDPS)*, 2022, pp. 368–378.
 - [108] S. Herbein, D. H. Ahn, D. Lipari, T. R. Scogland, M. Stearman, M. Grondona, J. Garlick, B. Springmeyer, and M. Taufer, “Scalable i/o-aware job scheduling for burst buffer enabled hpc clusters,” in *Proceedings of the 25th ACM Intl. Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 69–80. [Online]. Available: <https://doi.org/10.1145/2907294.2907316>

- [109] N. Fauzia, V. Elango, M. Ravishankar, J. Ramanujam, F. Rastello, A. Rountev, L.-N. Pouchet, and P. Sadayappan, "Beyond reuse distance analysis: Dynamic analysis for characterization of data locality potential," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 4, dec 2013. [Online]. Available: <https://doi.org/10.1145/2541228.2555309>
- [110] Z. Tang, M. Liu, A. Ammar, K. Li, and K. Li, "An optimized mapreduce workflow scheduling algorithm for heterogeneous computing," *The Journal of Supercomputing*, vol. 72, pp. 2059–2079, 2016.
- [111] K. Wang, X. Zhou, T. Li, D. Zhao, M. Lang, and I. Raicu, "Optimizing load balancing and data-locality with data-aware scheduling," in *2014 IEEE Intl. Conf. on Big Data (Big Data)*. IEEE, 2014, pp. 119–128.
- [112] K. Ranganathan and I. Foster, "Decoupling computation and data scheduling in distributed data-intensive applications," in *Proceedings 11th IEEE International Symposium on High Performance Distributed Computing*. IEEE, 2002, pp. 352–358.
- [113] A. Gaineru, G. Aupy, A. Benoit, F. Cappello, Y. Robert, and M. Snir, "Scheduling the i/o of hpc applications under congestion," in *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 2015, pp. 1013–1022.
- [114] J. Carretero, E. Jeannot, G. Pallez, D. E. Singh, and N. Vidal, "Mapping and scheduling hpc applications for optimizing i/o," in *Proceedings of the 34th ACM International Conference on Supercomputing*, 2020, pp. 1–12.

APPENDIX

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

A. Paper's Main Contributions

- C_1 We present a *workflow-centric methodology* to characterize, visualize, and analyze I/O behavior in HPC workflows by correlating workflow structure (tasks, stages, producer-consumer relationships) with low-level I/O metrics (dataflow size, access type, and operation count).
- C_2 We apply this methodology to six diverse real-world HPC workflows and conduct a holistic analysis of their producer-consumer dataflow patterns, revealing workflow-level I/O behaviors including file reuse, access asymmetry, and data-exchange bottlenecks.
- C_3 We derive 16 I/O-specific findings that expose limitations of current workflow execution and scheduling models, and identify new opportunities for I/O-aware scheduling, task placement, and storage selection in HPC systems.

B. Computational Artifacts

All artifacts are hosted in a single GitHub repository: <https://github.com/candiceT233/dataflow-for-io-aware>.

- A_1 Characterization and I/O-analysis framework, including trace-processing scripts and analysis notebooks.
- A_2 Raw workflow I/O traces for selected workflows: 1000 Genomes traces under `1000-genome/` and DeepDriveMD traces under `ddmd/`. Aggregated CSV inputs for all workflows are under `workflow_data/`.
- A_3 Analysis and visualization notebooks (`wf*.ipynb`) that generate the multi-panel figures (Figs. 1–6), with outputs in per-workflow `*_plots/` directories.

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3	Figs. 1–6, all findings
A_2	C_1, C_2, C_3	Traces for Figs. 2, 5
A_3	C_1	Figs. 1–6

C. Computational Artifact A_1

Relation To Contributions

A_1 is the main implementation of the workflow-centric I/O characterization methodology. It contains code and notebooks that ingest Datalife JSON traces (`*.datalife.json`, `*.r_blk_trace.json`, `*.w_blk_trace.json`), build task-file dataflow graphs, compute workflow- and I/O-level metrics, and generate the paper's figures. Major components include: (1) preprocessing scripts that convert raw traces into per-task/per-file I/O statistics in CSV form; and (2) analysis and visualization notebooks (`wf*.ipynb`) that produce 3D/2D plots grouped by task, file type, operation, and stage, with outputs saved to per-workflow `*_plots/` directories. Supports C_1 , C_2 , and C_3 .

Expected Results

Running the pipeline on provided traces yields per-task and per-file I/O metrics, producer-consumer dataflow graphs, and aggregate statistics matching the paper. The reproduced results demonstrate that workflow structure drives I/O behavior, that file reuse and asymmetric read/write patterns are common, and that these behaviors expose opportunities for I/O-aware scheduling.

Expected Reproduction Time (in Minutes)

- **Setup:** less than 1 hour to clone the repo, run the environment setup script, and optionally build Datalife.
- **Execution:** about 10 minutes to run the analysis and plotting notebooks on the provided traces.
- **Analysis:** 30 min to 1 hour to inspect generated PDFs and compare with the paper's figures and findings.

Artifact Setup (incl. Inputs)

Hardware: Commodity workstation or laptop; multi-core CPU, ≥ 8 GB RAM, ≥ 100 GB disk. No special interconnect required. Trace collection requires HPC access with Datalife.

Software: Python ≥ 3.10 ; pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, networkx, Jupyter (see `requirements.txt`). Datalife (<https://github.com/pnnl/datalife>) needed only for new trace collection. Tested on Linux.

Datasets / Inputs: Complete traces are provided for 2 of the 6 workflows: 1000 Genomes under `1000-genome/` and DeepDriveMD under `ddmd/`. Aggregated CSV inputs for all workflows are under `workflow_data/`. To obtain traces for the remaining four workflows, run them under Datalife with the original inputs.

Installation and Deployment: (1) `git clone https://github.com/candiceT233/dataflow-for-io-aware`; (2) set up the Python environment: `./setup_python_env.sh` (or `./setup_python_env.sh .venv-analysis` for a custom venv name); (3) optionally build Datalife for new trace collection: `git clone https://github.com/pnnl/datalife`, then `mkdir build && cd build && cmake .. && make`; (4) open the desired `wf*.ipynb` notebook, set `WORKFLOW_KEY` to the target workflow (e.g., "pyflex", "1kgenome", "ddmd"), and run all cells.

Artifact Execution

The experiment workflow consists of four tasks: T_1 (ingest and parse traces) $\rightarrow T_2$ (build dataflow graphs, compute per-task/per-file metrics) $\rightarrow T_3$ (aggregate metrics to CSV) $\rightarrow T_4$ (generate plots). Open the appropriate `wf*.ipynb` notebook, set `WORKFLOW_KEY` (e.g., "1kgenome", "ddmd"), and

run all cells to produce CSVs and plots. One trace set per workflow; no repetitions needed for qualitative reproduction.

Artifact Analysis (incl. Outputs)

Outputs: (1) CSV files with per-task/per-file I/O metrics (columns include operation, taskName, fileName, dataflow size, opCount, bandwidth, stageOrder); (2) PDF plots per workflow in the corresponding `*_plots/` directory, corresponding to Figs. 1–6. Verify by comparing cluster shapes, stage-wise trends, and relative positions; minor numerical variation is acceptable.

D. Computational Artifact A_2

Relation To Contributions

A_2 provides raw Datalife traces for 1000 Genomes (Fig. 2) and DeepDriveMD (Fig. 5), supporting C_1 , C_2 , and C_3 . A_3 provides the visualization notebooks that generate Figs. 1–6, supporting C_1 . Both reside in the same repository as A_1 .

Expected Results

A_2 enables reproducing the analysis for Figs. 2 and 5 and related findings. A_3 enables regenerating all multi-panel figures.

Expected Reproduction Time (in Minutes)

Same as A_1 ; trace data is included and notebooks run in less than 10 minutes in total.

Artifact Setup (incl. Inputs)

Hardware: Same as A_1 .

Software: Same as A_1 .

Datasets / Inputs: A_2 : traces in `1000-genome/` and `ddmd/`; aggregated CSVs for all workflows in `workflow_data/`. A_3 : `wf*.ipynb` notebooks with outputs in `*_plots/` directories.

Installation and Deployment: Same as A_1 .

Artifact Execution

A_2 : traces are consumed by A_1 's T_1 ; no separate workflow. A_3 : corresponds to $T_3 \rightarrow T_4$ in A_1 .

Artifact Analysis (incl. Outputs)

A_2 : input data with no standalone output. A_3 : PDF plots matching Figs. 1–6.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Clone the repository and set up the environment as described in A_1 's Installation and Deployment section. Ensure `1000-genome/`, `ddmd/`, `workflow_data/`, and the `wf*.ipynb` notebooks are present. No compilation of the analysis code is needed; Jupyter runs the notebooks.

Artifact Execution

Step 1: Open `wf2_IO_profile_analysis_1kgenome.ipynb` (or the corresponding 3D notebook), set `WORKFLOW_KEY="1kgenome"`, and execute all cells. Step 2: Repeat for DDMD using the `wf5_*` notebook with `WORKFLOW_KEY="ddmd"`. Both notebooks use preprocessed CSVs from `workflow_data/` and traces from the included Datalife trace directories.

Artifact Analysis (incl. Outputs)

Expected: (1) CSVs with I/O metrics per task/file/operation; (2) PDFs in the corresponding `*_plots/` directories matching Figs. 1–6. The same qualitative patterns (clusters, stage-wise variation, file reuse) should appear. Minor numerical or layout differences are acceptable.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Optional: re-run a workflow (e.g., 1000 Genomes) under Datalife (<https://github.com/pnnl/datalife>) on an HPC system to collect new traces, then run the same pipeline as in Evaluation 1.

Artifact Execution

Task 1: Deploy and run the workflow under Datalife to produce new JSON traces. Task 2: Process traces with the appropriate `wf*.ipynb` notebook. Task 3: Generate plots by running all cells. Dependencies: Task 2 $\rightarrow$ Task 3; Task 1 depends on local HPC setup.

Artifact Analysis (incl. Outputs)

New CSVs and PDFs with the same structure as Evaluation 1. Absolute values may differ due to hardware; qualitative I/O patterns should match the paper's conclusions.